

EXERCÍCIO 03 – GEORREFERENCIAMENTO COM MONGODB E GEOJSON

Implementação e Gerenciamento de Bancos de Dados NoSQL

Código no GitHub: github.com/joseniltoncirne/Implementacao-e-Geren-de-banco-de-dados/tree/main/exercicio03-geojson

O que o script faz

O script coletador_geojson.py pega a lista de postinhos de saúde (UBS) do Brasil inteiro no site de dados abertos do governo e guarda no MongoDB, no banco geodados, coleção unidades_saude. Ele faz isso em 4 passos:

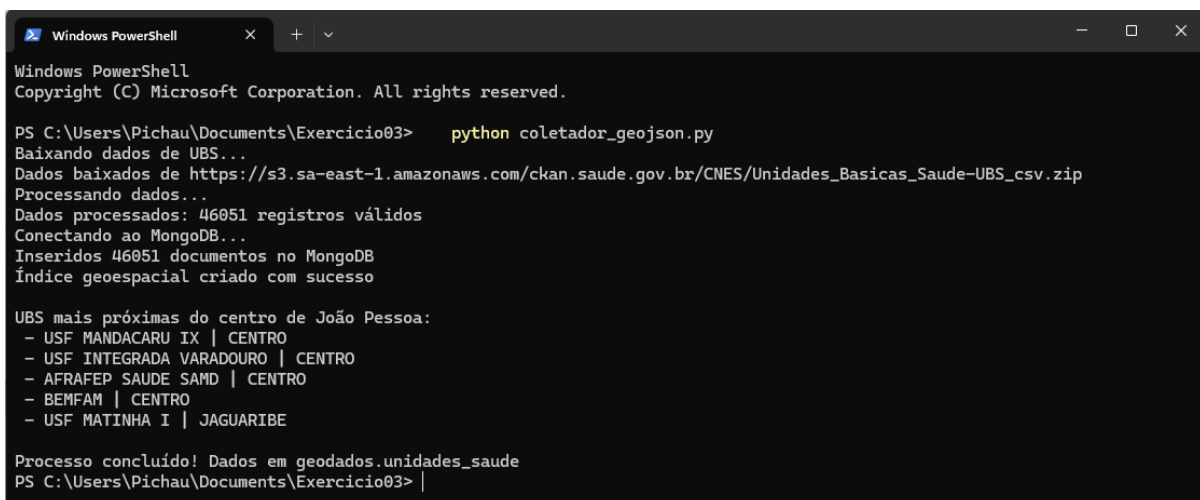
- **Baixa a lista:** o governo entrega o arquivo zipado. O script baixa e já abre o zip sozinho.
- **Dá uma limpada:** joga fora os postos que não têm localização e arruma os números da localização que vieram com vírgula em vez de ponto.
- **Salva no formato GeoJSON:** é um jeito padrão de guardar lugares. Cada posto fica com o seu ponto no mapa e com as informações dele, como nome, rua e bairro.
- **Cria o índice de localização:** é o que deixa o MongoDB responder perguntas como "quais postos ficam perto daqui?" de forma rápida.

Antes de salvar, o script apaga o que já estava lá. Assim dá para rodar quantas vezes quiser sem ficar nada repetido.

1. Instalando o que precisa e rodando

```
python -m pip install -r requirements.txt
python coletador_geojson.py
```

O script baixou a lista do governo e salvou 46.051 postos de saúde. No final, ele faz um teste: pergunta quais são os 5 postos mais perto do Centro de João Pessoa. Todos os que aparecem ficam no Centro, no Varadouro e em Jaguaribe, ou seja, a busca por localização está funcionando.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

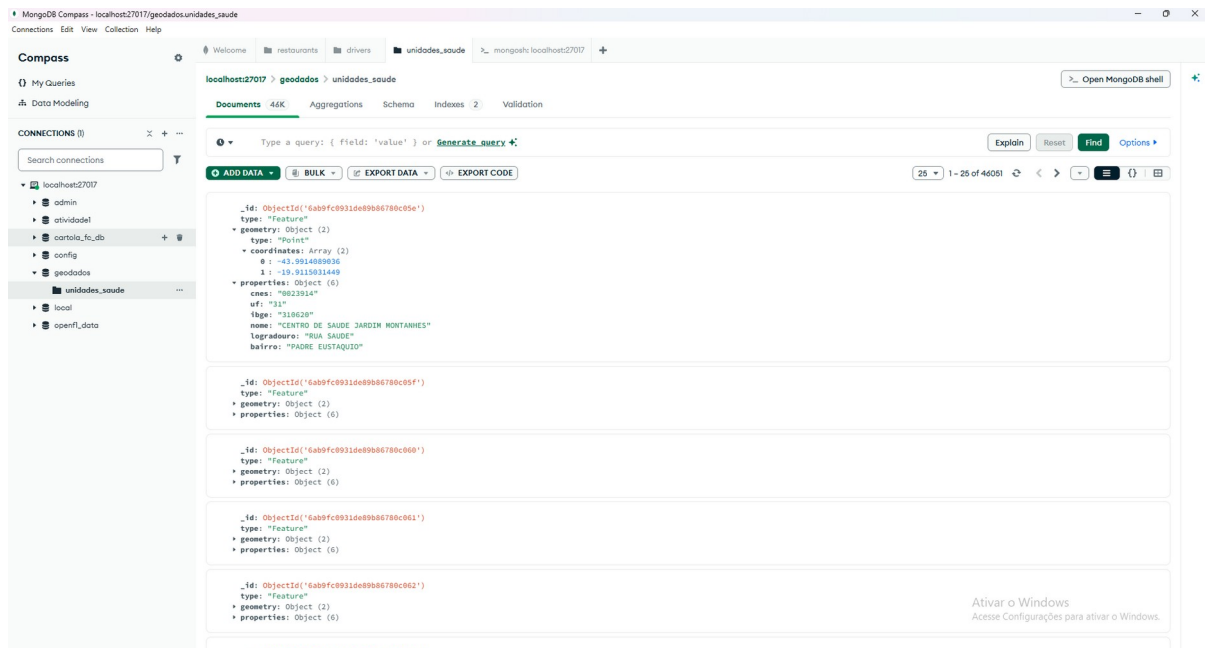
PS C:\Users\Pichau\Documents\Exercicio03> python coletador_geojson.py
Baixando dados de UBS...
Dados baixados de https://s3.sa-east-1.amazonaws.com/ckan.saude.gov.br/CNES/Unidades_Basicas_Saude-UBS_csv.zip
Processando dados...
Dados processados: 46051 registros válidos
Conectando ao MongoDB...
Inseridos 46051 documentos no MongoDB
Índice geoespacial criado com sucesso

UBS mais próximas do centro de João Pessoa:
- USF MANDACARU IX | CENTRO
- USF INTEGRADA VARADOURO | CENTRO
- AFRAFEP SAUDE SAMD | CENTRO
- BEMFAM | CENTRO
- USF MATINHA I | JAGUARIBE

Processo concluído! Dados em geodados.unidades_saude
PS C:\Users\Pichau\Documents\Exercicio03> |
```

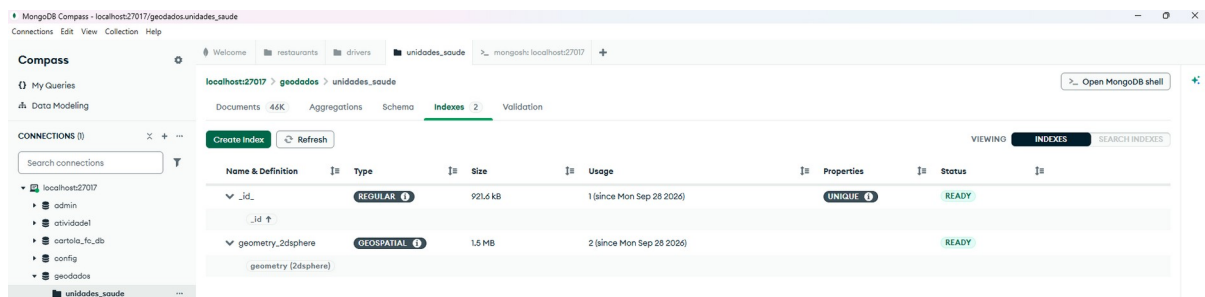
2. Como os postos ficaram guardados

Abrindo um dos postos dá para ver como ele ficou guardado: em geometry fica o ponto no mapa e em properties ficam as informações do posto (nome, rua, bairro). E mesmo depois de rodar o script mais de uma vez, continuam sendo 46.051 postos, nada repetido.



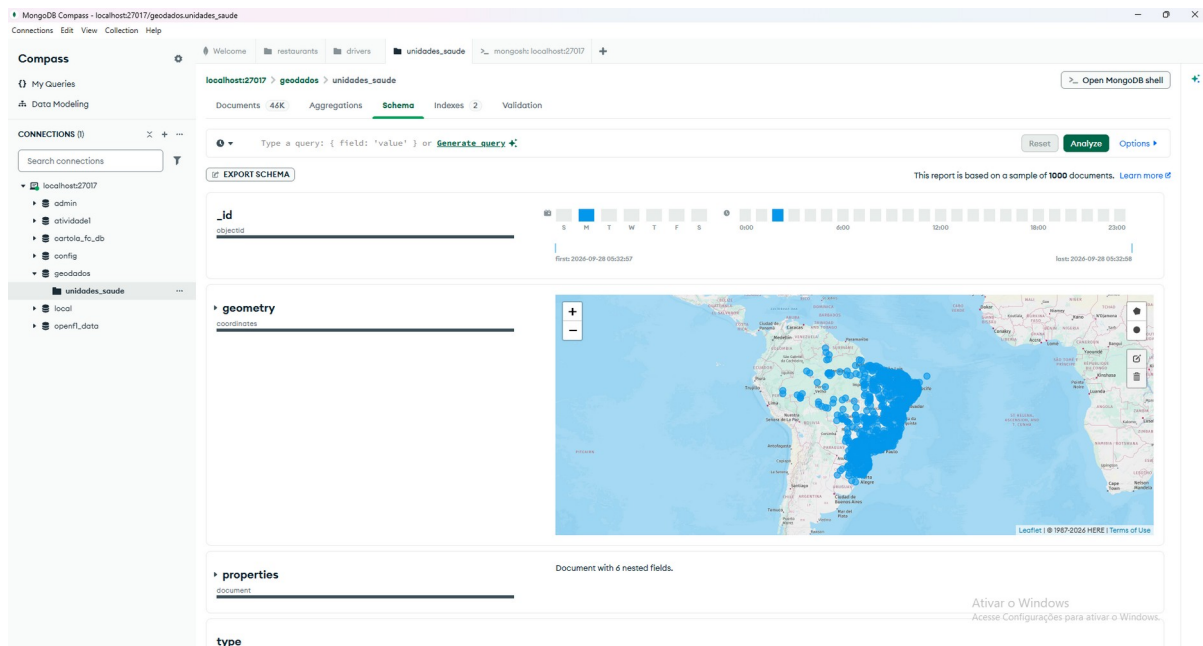
3. Índice de localização

Na aba Indexes aparece o índice de localização (geometry_2dsphere). O READY do lado quer dizer que ele já está funcionando.



4. Os postos no mapa

Na aba Schema, o Compass mostra os postos no mapa. Ele não desenha todos, só 1.000 deles, para o mapa não ficar pesado. Cada bolinha azul é um posto de saúde.



Código

coletador_geojson.py

```
import io
import os
import zipfile

import pandas as pd
import geopandas as gpd
import requests
from pymongo import MongoClient
from shapely.geometry import Point

URLS_DADOS = [
    "https://s3.sa-east-1.amazonaws.com/ckan.saude.gov.br/CNES/Unidades_Basicas_Saude-UBS_csv.zip",
    "https://s3.sa-east-1.amazonaws.com/ckan.saude.gov.br/UBS/ubs.csv",
    "https://s3.sa-east-1.amazonaws.com/ckan.saude.gov.br/PDA/UNIDADES_BASICAS_SAUDE/ubs.csv",
]
NOME_ARQUIVO = "ubs.csv"
MONGO_URI = "mongodb://localhost:27017/"
NOME_BANCO = "geodados"
NOME_COLECAO = "unidades_saude"

def baixar_dados():
    print("Baixando dados de UBS...")
    for url in URLS_DADOS:
        try:
            response = requests.get(url, timeout=60)
            response.raise_for_status()
            conteudo = response.content
            if zipfile.is_zipfile(io.BytesIO(conteudo)):
                with zipfile.ZipFile(io.BytesIO(conteudo)) as z:
                    nome_csv = next(n for n in z.namelist() if n.lower().endswith(".csv"))
                    conteudo = z.read(nome_csv)
            with open(NOME_ARQUIVO, "wb") as f:
                f.write(conteudo)
            print(f"Dados baixados de {url}")
            return True
        except requests.RequestException:
            print(f"Não deu para baixar de {url}")

    if os.path.exists(NOME_ARQUIVO):
        print(f"Usando o arquivo {NOME_ARQUIVO} da pasta")
        return True

    print(f"Baixe o CSV de UBS pelo navegador e salve como {NOME_ARQUIVO} nesta pasta")
    return False

def processar_dados():
    print("Processando dados...")
    try:
        df = pd.read_csv(NOME_ARQUIVO, sep=None, engine="python", dtype=str, encoding="utf-8-sig")
    except UnicodeDecodeError:
        df = pd.read_csv(NOME_ARQUIVO, sep=None, engine="python", dtype=str, encoding="latin-1")

    df.columns = [c.strip().lower() for c in df.columns]
    col_lat = next(c for c in df.columns if "latitude" in c)
    col_lon = next(c for c in df.columns if "longitude" in c)

    df["latitude"] = pd.to_numeric(df.pop(col_lat).str.replace(",", "."), errors="coerce")
    df["longitude"] = pd.to_numeric(df.pop(col_lon).str.replace(",", "."), errors="coerce")
    df = df.dropna(subset=["latitude", "longitude"])
    df = df[df["latitude"].between(-34, 6) & df["longitude"].between(-74, -28)]

    geometry = [Point(xy) for xy in zip(df["longitude"], df["latitude"])]
    gdf = gpd.GeoDataFrame(df, geometry=geometry, crs="EPSG:4674")
    print(f"Dados processados: {len(gdf)} registros válidos")
    return gdf
```

```

def salvar_no_mongodb(gdf):
    print("Conectando ao MongoDB...")
    collection = MongoClient(MONGO_URI)[NOME_BANCO][NOME_COLECAO]
    collection.delete_many({})

    propriedades = gdf.drop(columns=["latitude", "longitude", "geometry"])
    propriedades = propriedades.astype(object).where(propriedades.notna(), None)

    features = []
    for (_, row), props in zip(gdf.iterrows(), propriedades.to_dict("records")):
        features.append({
            "type": "Feature",
            "geometry": {"type": "Point", "coordinates": [row["longitude"], row["latitude"]]},
            "properties": props,
        })

    collection.insert_many(features)
    print(f"Inseridos {len(features)} documentos no MongoDB")

    collection.create_index([("geometry", "2dsphere")])
    print("Índice geoespacial criado com sucesso")
    return collection

def consulta_exemplo(collection):
    print("\nUBS mais próximas do centro de João Pessoa:")
    perto = collection.find({
        "geometry": {"$near": {
            "$geometry": {"type": "Point", "coordinates": [-34.8800, -7.1200]},
            "$maxDistance": 3000,
        }}
    }).limit(5)
    for doc in perto:
        print(" -", doc["properties"].get("nome"), "|", doc["properties"].get("bairro"))

def main():
    if not baixar_dados():
        return
    gdf = processar_dados()
    collection = salvar_no_mongodb(gdf)
    consulta_exemplo(collection)
    print(f"\nProcesso concluído! Dados em {NOME_BANCO}.{NOME_COLECAO}")

if __name__ == "__main__":
    main()

```

requirements.txt

```

requests
pandas
geopandas
pymongo
shapely

```